5.2 Android consumer test implementation

Now we can get to the main part of the chapter, which is implementing the contract tests for the data layer. We'll detail the implementation of the API repository and client hooks directly into the contract tests, which means that any change to the API implementation within the mobile app will automatically change the test implementation as well. Don't worry if you aren't familiar with the inner workings of Android native mobile apps. The important thing to take from the movie API client is that the client and repository implementation used within the test is the same as that used within the app. The only difference is that the URL has been replaced by the Pact mock server. This provides a significant benefit of implementing contract testing, linking back to some of the challenges we described in the introduction around API documentation becoming outdated. For example, the mobile app could filter the data returned from the movies API within the query parameters to the most recent 50 films to improve performance. To do that, the mobile app implementation can change the request in its code, and the Pact test automatically picks up the change. Therefore, an integration test is still recommended to test the real URL to verify that the configuration and authorization have been set up correctly.

Let's start writing consumer contract tests. As we're using Kotlin and JUnit 5, there are annotations that help us set up the test. The Pact consumer test sits along with the unit tests within the test projects, as shown in figure 5.2.

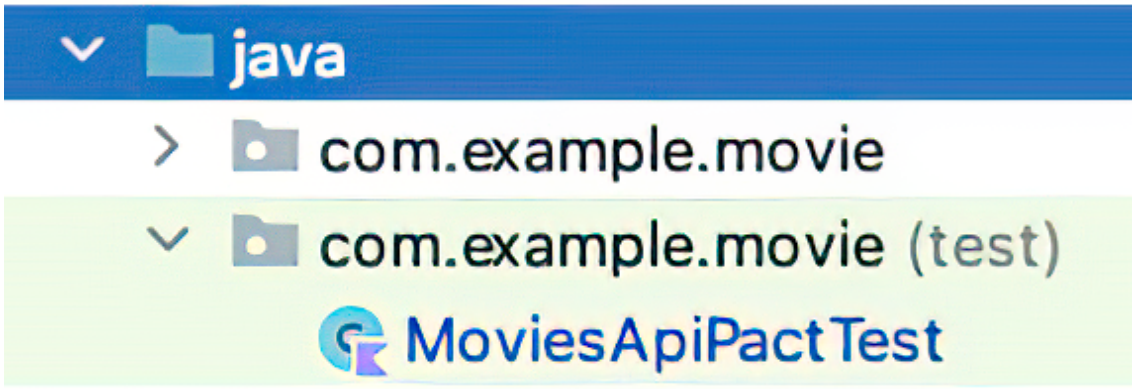


Figure 5.2 Android movie app folder structure showing the application com.example.movie folder. Under test folders, MoviesApiPactTest is highlighted.